Technical Documentation

dm-Verity Integrity Test Suite

Team Oriented Project

Group A

Linux Kernel Extension for DM-Verity Authentication, Verification and Setup

Date: **NOVEMBER 2025**

Cooperation with:

BMW Car IT, Ulm

Eva-Helena Zorman

Tobias Kaufmann

Philipp Graf

1. Introduction

1.1. Purpose

This document provides technical specifications for the integrity_tests.sh script, a dedicated robustness validation suite for the Linux kernel's dm-verity parser. The script's purpose is to systematically generate corrupted disk images to test the kernel subsystem's resilience against malicious or malformed metadata.

The tests focus on validating: input sanitization, bounds checking, integer/buffer overflow protection, and graceful error handling under adversarial conditions.

1.2. Background and Testing Methodology

This script operates on a pre-verified rootfs.img and corrupts specific binary structures within the dm-verity metadata (the locator, header, or signature). The resulting corrupted image is then intended to be booted in an isolated environment (like QEMU) to observe the kernel's response, verifying that it fails securely without crashing or yielding control to attackers.

1.3. Output and Result Location

The primary result of executing this script is a new, corrupted disk image file, which must be used for testing.

- **Test Image Location:** The corrupted image is created in the same directory as the source image, following the naming convention: `rootfs.<TEST_MODE>.test.img` (e.g., rootfs.meta1.test.img).
- **Kernel Behavior:** The final and most important result is the kernel's logged behavior during the boot process (observed within the QEMU environment), which is compared against the expected security failure.

2. Script Specification: integrity_tests.sh

2.1. Overview

The script is a Bash wrapper that orchestrates complex binary corruption using helper functions, primarily written in Python, for precise byte manipulation and offset calculation. It ensures strict error handling using `set -euo pipefail`.

2.2. Complete List of Test Modes

The script defines and executes the following seven specific corruption modes, directly corresponding to potential dm-verity parser vulnerabilities:

Test Mode	Corruption Target	Security Check
meta1	Flip single byte in the dm-verity header (at offset +64).	Header checksum validation, structural integrity checks.
sig1	Flip single byte in the PKCS#7 signature blob.	Cryptographic signature verification, ASN. parsing robustness.
int_overflow	Set locator fields to values that trigger integer wrap-around.	32/64 -bit integer overflow detection, bounds validation.
buf_overflow	Set metadata length field to maximum value (0xFFFFFFFF).	Buffer size validation, memory allocation limits.
trunc_meta	Claim metadata region extends beyond physical disk capacity.	Bounds checking, truncated read error handling.
bad_offsets	Set metadata/signature offsets beyond the disk end boundary.	Offset validation, out-of-bounds access prevention.
sanitize	Replace locator with completely random field values.	Magic number validation, input field sanitization.

2.3. Python Helper Functions: Purpose and Mechanism

The core logic of the validation suite relies on helper functions written in Python due to the necessity of performing **precise, low-level binary manipulation** of the disk image. Bash and standard Linux utilities like dd are insufficient for the complex tasks of reading and rewriting structured binary data at exact offsets within a disk image.

The Python standard library, particularly the **struct** module (used implicitly here for packing/unpacking binary data) and direct **file-like access to the block device**, allows the script to:

- **Parse Binary Structures:** Read the VLOC footer's binary data and unpack the exact 32-bit and 64-bit offsets and lengths (Q for 64-bit, I for 32-bit) used by the kernel.
- **Targeted Corruption:** Seek to a specific byte offset (calculated in the shell and passed to Python) and perform a byte-by-byte modification (like the XOR flip in python_flip_byte) without affecting surrounding disk blocks.

- **Privileged I/O:** Run under sudo to gain direct read/write access to the raw block device ($LOOP) established by losetup.

These functions abstract the complexity of working with the on-disk dm-verity format, making the overall test logic cleaner and more robust.

The script relies entirely on these three Python functions to perform all read and write operations directly on the disk image through the loop device ($LOOP):

1. python_read_locator (Geometry and Offset Reading)

This function reads the VLOC footer to parse and expose the original metadata/signature offsets and lengths:

```
python_read_locator() {
    cat <<'PY'
import sys, struct, os
loop = sys.argv[1]
size = int(sys.argv[2])
loc_off = size - 4096

with open(loop, 'rb', buffering=0) as f:
    f.seek(loc_off)
    blk = f.read(32)

print("DEBUG: Raw locator bytes:", blk.hex(), file=sys.stderr)

try:
    magic, version, meta_off, meta_len, sig_off, sig_len = struct.unpack('<IIQIQI', blk[:32])
except struct.error as e:
    print(f"ERROR: Failed to unpack locator: {e}", file=sys.stderr)
    sys.exit(2)

print(f"DEBUG: magic=0x{magic:08x}, version={version}", file=sys.stderr)
print(f"DEBUG: meta_off={meta_off} (0x{meta_off:x}), meta_len={meta_len}", file=sys.stderr)
print(f"DEBUG: sig_off={sig_off} (0x{sig_off:x}), sig_len={sig_len}", file=sys.stderr)

if magic == 0x564C4F43:  # VLOC
    print(f"{meta_off} {meta_len} {sig_off} {sig_len} {loc_off}")
else:
    print(f"ERROR: Invalid VLOC magic - got 0x{magic:08x}, expected 0x564C4F43", file=sys.stderr)
    sys.exit(1)
PY
}
```

2. python_flip_byte (Single-Byte Corruption)

This function is used for meta1 and sig1 tests, toggling a single bit at a precise offset to simulate minimal corruption:

```
python_flip_byte() {
    local offset=$1
    cat <<PY
import os, sys
p = os.environ['LOOP']
off = $offset
with open(p, 'r+b', buffering=0) as f:
    f.seek(off)
    b = f.read(1)
    if len(b) != 1:
        sys.stderr.write(f"ERROR: could not read 1 byte at {off}\n")
        sys.exit(2)
    f.seek(off)
    written = f.write(bytes([b[0] ^ 0x01]))
    if written != 1:
        sys.stderr.write("ERROR: short write when flipping byte\n")
        sys.exit(2)
    f.flush()
    os.fsync(f.fileno())
print(f"flipped 1 byte at {off}")
PY
}
```

3. python_corrupt_locator (Offset/Length Corruption)

This function rewrites the VLOC footer with arbitrary (malicious) offset and length values to trigger bounds and overflow checks, used by int_overflow, buf_overflow, trunc_meta, and bad_offsets:

```
python_corrupt_locator() {
    local meta_off=$1 meta_len=$2 sig_off=$3 sig_len=$4
    cat <<PY
import struct, os, sys
loop_dev = os.environ['LOOP']
loc_off = int(os.environ['LOCATOR_OFFSET'])

with open(loop_dev, 'rb', buffering=0) as f:
    f.seek(loc_off)
    locator_data = f.read(24)
    if len(locator_data) < 24:
        sys.stderr.write('ERROR: short locator read\n')
        sys.exit(2)
    magic, version, orig_meta_off, orig_meta_len, orig_sig_off, orig_sig_len = struct.unpack('<IIIIII', locator_data)

print('Original values:')
print(f'  meta_off: {orig_meta_off}, meta_len: {orig_meta_len}')
print(f'  sig_off:  {orig_sig_off}, sig_len:  {orig_sig_len}')

print('Corrupted values:')
print(f'  meta_off: {$meta_off} (0x{$meta_off:08x})')
print(f'  meta_len: {$meta_len} (0x{$meta_len:08x})')
print(f'  sig_off:  {$sig_off} (0x{$sig_off:08x})')
print(f'  sig_len:  {$sig_len} (0x{$sig_len:08x})')

new_locator = struct.pack('<IIIIII', magic, version, $meta_off, $meta_len, $sig_off, $sig_len)
with open(loop_dev, 'r+b', buffering=0) as f:
    f.seek(loc_off)
    written = f.write(new_locator)
    if written != len(new_locator):
        sys.stderr.write('ERROR: short write when updating locator\n')
        sys.exit(2)
    f.flush(); os.fsync(f.fileno())

print('SUCCESS: Locator corrupted')
PY
}
```

3. Test Execution and Execution Safeguards

3.1. Execution Workflow

The script handles image preparation, execution, and cleanup:

1. **Image Preparation:** The image is attached to a loop device ($LOOP) using sudo losetup -f --show.
2. **Corruption:** The selected test mode (e.g., meta1) calls the corresponding Python helper with calculated offsets (e.g., META_OFFSET + 64) to perform the write.
3. **Cleanup:** The cleanup trap ensures the loop device is always detached upon script exit.

3.2. Execution Safeguards

The script provides a crucial **non-destructive default** to prevent data loss:

- **Default Behavior:** It copies the original image to a new test file (`rootfs.<TEST_MODE>.test.img`) before corruption.

- **Destructive Options:** The `--inplace` option allows direct modification of the source image but is guarded by user confirmation and the availability of `--backup`/`--restore` mechanisms to manage the original file.

4. Test Outcome Analysis

This section details the expected secure failure mechanism for each test mode when the corrupted image is booted by the kernel. The goal is always to ensure the kernel rejects the disk and terminates the boot process cleanly without a panic or uncontrolled memory access.

- **meta1**
    - Expected Kernel Behavior: Rejection. Kernel fails to parse the structural integrity of the dm-verity metadata header.
    - Reason for Failure (The "Why"): The single byte flip causes the header checksum or internal structural parameters (like block size) to become invalid, proving the metadata was tampered with after signing.
- **sig1**
    - Expected Kernel Behavior: Rejection. dm-verity creation fails due to a cryptographic mismatch.
    - Reason for Failure (The "Why"): The PKCS#7 signature verification process will fail because the SHA256 hash of the PKCS#7 structure (which now contains a flipped bit) no longer matches the digital signature provided by the trusted key.
- **int_overflow**
    - Expected Kernel Behavior: Rejection. Kernel validation routines detect an arithmetic overflow.
    - Reason for Failure (The "Why"): Setting fields like offsets or lengths to near-maximum 32-bit or 64-bit values causes calculation logic (e.g., offset + length) to wrap around, resulting in a physically impossible address.
- **buf_overflow**
    - Expected Kernel Behavior: Rejection. Kernel returns an error on memory allocation limits.
    - Reason for Failure (The "Why"): The enormous length field (0xFFFFFFFF) triggers kernel internal checks against maximum allowable memory allocation sizes, preventing a catastrophic kernel memory buffer overflow or a system crash due to exhaustion of resources.
- **trunc_meta**
    - Expected Kernel Behavior: Rejection. Kernel returns an EIO (Input/Output Error) or EINVAL.

- Reason for Failure (The "Why"): The calculated bounds check fails, as the dm-verity parser attempts to read the metadata from a region that logically extends past the total physical size of the disk image, violating I/O boundaries.

- **bad_offsets**
  - Expected Kernel Behavior: Rejection**.** Kernel rejects the offsets before reading.
  - Reason for Failure (The "Why"): The parser performs an initial sanity check: the start address of the metadata (`$META\_OFFSET`) must be less than the total disk size (`$DISK\_BYTES`). The corrupted, large offset fails this primary bounds check.
- **sanitize**
  - Expected Kernel Behavior: Rejection. Failure upon initial magic number read.
  - Reason for Failure (The "Why"): The VLOC footer is intended to start with the magic number 0x564C4F43 ('VLOC'). The random data fails this basic security check immediately, preventing the parser from attempting to interpret the garbage fields that follow.

End of Document